Plan: Facial Expression Recognition with Real-Time Analysis

TL;DR

Build a complete FER system in 2-3 days by **dividing work into 4 independent modules** that can run in parallel: (1) data preprocessing pipeline, (2) feature extraction & model training, (3) real-time inference engine with visualization, (4) report & documentation. Use transfer learning or pre-trained models to stay on timeline. The project will showcase **emotion intensity scoring, confidence visualizations, and live webcam processing**—making it stand out while meeting all course requirements.

Steps

Phase 1: Setup & Data Pipeline (Parallel — All can start Day 1)

1. **Person A - Data & Preprocessing Module**
    - Download FER2013 dataset using kagglehub (already in your notebook)
    - Implement preprocessing pipeline:
        - Grayscale image validation
        - Gaussian blur for noise removal
        - **CLAHE (Contrast Limited Adaptive Histogram Equalization)** for enhancement (implement from scratch using histogram bins for course credit, or use `cv2.createCLAHE()`)
        - Bilateral filtering to preserve edges while smoothing
        - Output: Clean, enhanced image tensors saved to disk
    - Split dataset into train (80%), validation (10%), test (10%)
    - Document preprocessing flow with before/after visualizations
2. **Person B - Feature Extraction & Classifier Training**

- Face detection: Use `dlib` or `mediapipe` for face ROI extraction (requirement: segmentation component)
- Feature extraction from face ROI:
  - **Primary**: HOG (Histogram of Oriented Gradients) via scikit-image or implement manually
  - **Secondary**: LBP (Local Binary Patterns) for texture analysis
  - **Tertiary**: Edge-based features (Canny + histogram of edges)
  - Combine features into feature vectors
- Train 2-3 classifiers in parallel:
  - SVM with RBF kernel
  - Random Forest (for confidence calibration)
  - KNN as fallback/ensemble component
- Store trained models + scalers for inference

3. **Person C - Real-Time Inference Engine & Visualization**
   - Build modular inference pipeline:
     - Accept image/webcam frame
     - Apply preprocessing (same as Person A)
     - Extract features (same as Person B)
     - Run all 3 classifiers for ensemble prediction
   - Create **multi-faceted analysis output**:
     - Emotion classification (primary label)
     - Confidence scores (per-class probabilities)
     - **Emotion intensity** (normalized 0-1 scale per emotion)
     - Ensemble agreement score
   - Implement Matplotlib/Plotly visualization:
     - Bar chart: emotion probabilities + confidence intervals
     - Real-time webcam window with overlay (bounding box, emotion label, confidence)

4. **Person D - Report, Documentation & Integration**
    - Write 4-page report:
        - **Abstract**: System overview, uniqueness (real-time + intensity analysis)
        - **Problem Definition**: FER challenges (lighting, pose, individual differences)
        - **Objectives**: Build robust end-to-end FER with real-time capability
        - **Methodology**: Detail each stage (preprocessing, segmentation/feature extraction, classification)
        - Team names and IDs in table
    - Create main orchestrator notebook that:
        - Imports all modules (A, B, C)
        - Demonstrates: static image inference → visualization
        - Demonstrates: live webcam real-time processing
        - Includes sample results and metrics (accuracy, confusion matrix on test set)
    - Organize code into clean functions/classes for modularity

---

Phase 2: Testing & Refinement (Day 2-3, Parallel)

1. **Person A + B**: Run training on full dataset, capture metrics (accuracy, precision, recall per emotion)
2. **Person C**: Test inference pipeline on validation set; ensure visualizations are clear and responsive
3. **Person C**: Build live webcam demo; test latency (~30 FPS target)
4. **Person D**: Integrate all results into final notebook; verify all cells run end-to-end
5. **All**: Validate that all course requirements are met (preprocessing ✓, segmentation ✓, feature extraction ✓, classification ✓)

Relevant Files

- Image_Processing_Project.ipynb — Reuse data loading & colorization cells as optional enhancement; core work replaces coloring section
- Create: `preprocessing.py` (Person A) — noise removal, histogram equalization, filtering functions
- Create: `feature_extraction.py` (Person B) — HOG, LBP, edge features; classifier training
- Create: `inference.py` (Person C) — model loading, ensemble prediction, visualization code
- Create: `REPORT.md` (Person D) — course-required report
- Create: `main.ipynb` (Person D) — orchestrator notebook tying all modules together

---

Verification

1. **Data Pipeline** ✓
   - Verify preprocessing improves image clarity (before/after comparison on 5 test images)
   - Confirm train/val/test splits are balanced across emotion classes
2. **Model Training** ✓
   - Test set accuracy > 50% (baseline for FER2013 with standard features; challenge: achieve 60%+)
   - Generate confusion matrix showing which emotions are confused
   - Compare all 3 classifiers; confirm ensemble outperforms individual models
3. **Real-Time System** ✓
   - Run on webcam feed for 30 seconds; verify detects and classifies faces
   - Confirm emotion intensity scores sum to 1.0 (normalized)
   - Visualize confidence intervals; ensure they reflect model uncertainty
4. **Course Requirements** ✓
   - Preprocessing section: Document each technique used + computational effect

- Segmentation: Face ROI extraction confirmed
- Feature extraction: HOG + LBP + edge features clearly explained
- Classification: 3+ classifiers implemented, ensemble reasoning documented
- Report: All 4 required sections present, team IDs listed

5. **Integration** ✓
   - Main notebook runs without errors from top to bottom
   - All visualizations render correctly
   - Live demo completes without crashes

Decisions & Scope

Included:

- FER2013 dataset (full training)
- Custom webcam image augmentation (live demo)
- Multi-classifier ensemble with confidence scoring
- Emotion intensity analysis (normalized per-emotion probability)
- Real-time visualization with bounding boxes
- Course-required report with full methodology

Excluded (to stay on 2-3 day timeline):

- Deep learning models (too slow to train on 2-3 day deadline)
- Complex data augmentation (use raw FER2013 only)
- Extensive hyperparameter tuning (use sensible defaults)
- Video processing (single frame inference only; real-time via webcam frame-by-frame)

Technical Decisions:

- **Libraries**: OpenCV (preprocessing, face detection), scikit-learn (classifiers), scikit-image (HOG/LBP), NumPy/Pandas (data handling), Matplotlib (visualization)

- **Face detector**: Use mediapipe (faster, more reliable) or dlib (more academic credibility)
- **Feature combo**: HOG (industry standard) + LBP (texture) + edge histogram (robustness)
- **Ensemble**: Soft voting (average probabilities) for confidence calibration

---

Further Considerations

1. **Face Detection Library Choice**
   - Option A: **mediapipe** (fast, lightweight, less code) → Better for 2-3 day timeline
   - Option B: **dlib** (more academic, better explainability) → Better for report credibility
   - **Recommendation**: Start with mediapipe for speed; if time permits, compare both in report
2. **Emotion Intensity Interpretation**
   - **Clarification needed**: Should intensity reflect confidence from classifier, or a separate "strength of emotion" metric?
   - **Current approach**: Use softmax probabilities as intensity (0-1 per emotion class)
   - **Alternative**: Post-process probabilities to estimate intensity (e.g., max probability = intensity; others scaled accordingly)
3. **Real-Time Performance vs. Accuracy Trade-off**
   - **Decision**: If inference is slow (<15 FPS), downsize images or reduce feature dimensions
   - **Pre-decision**: Use 224×224 faces (standard for feature extraction)

Team Assignment Summary

Person	Role	Deliverables
A	Data & Preprocessing	`preprocessing.py`, train/val/test splits, before/after visualizations
B	Feature Extraction & Training	`feature_extraction.py`, trained classifiers, metrics (accuracy/confusion matrix)

Person	Role	Deliverables
C	Real-Time Engine & Visualization	`inference.py`, webcam demo, confidence + intensity visualizations
D	Report & Integration	`REPORT.md`, orchestrator `main.ipynb`, all modules working together

Parallelization: All 4 can start Day 1. Person D waits for output from A/B/C (ready by EOD Day 1-2). Final 8 hours = integration + polish.